

http://algs4.cs.princeton.edu

3.3 BALANCED SEARCH TREES

- *2-3 search trees*
- *red-black BSTs*
- *B-trees*

Symbol table

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
goal	$\log N$	$\log N$	$\log N$	$\log N$	$\log N$	$\log N$	✓	`compareTo()`

Sfida. Garantimi i performancës.

Kjo ligjëratë. 2-3 trees, left-leaning red-black BSTs, B-trees.



http://algs4.cs.princeton.edu

3.3 Balanced Search Trees

- *2-3 search trees*
- *red-black BSTs*
- *B-trees*

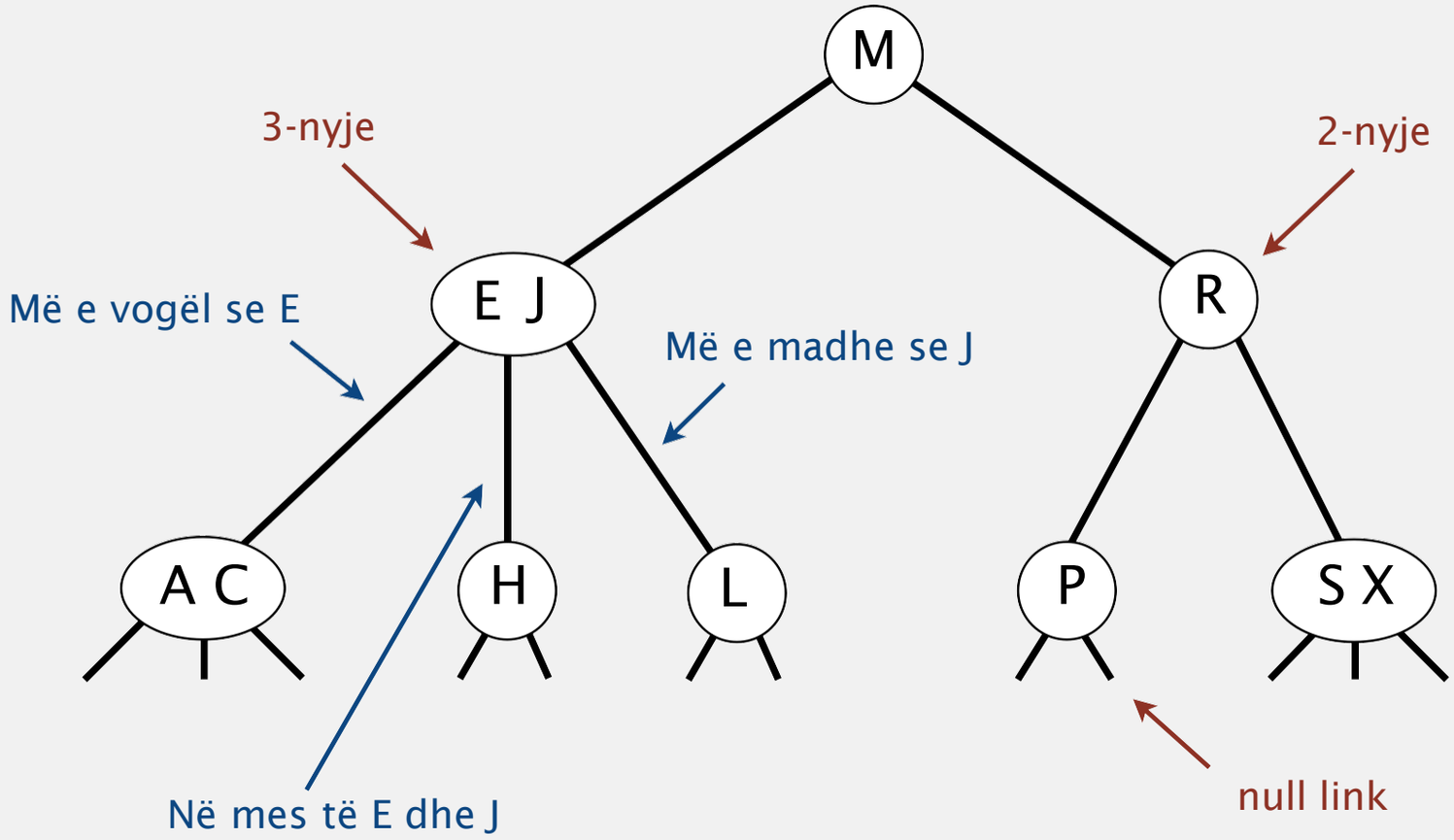
2-3 tree

Lejon 1 ose 2 çelësa për nyje.

- 2-node: një çelës, dy fëmijë.
- 3-node: dy çelësa, tre fëmijë.

Rradhitje simtrike. Inorder traversal jep çelësat e rradhitur nga më i vogli deri tek më i madhi.

Balancimi perfekt. Çdo rrugë prej root deri tek null link-u ka length të njejtë.



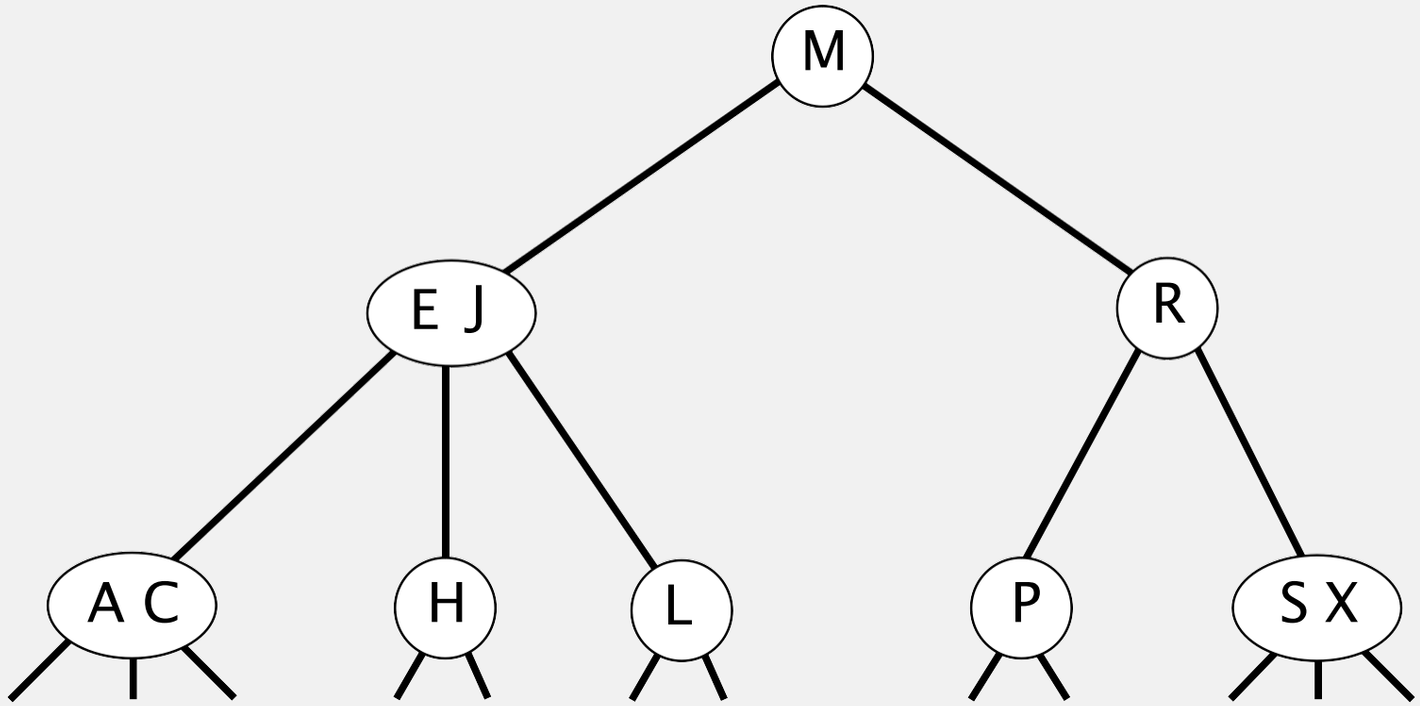
2-3 tree demo

Search.

- Krahasohet search çelësi me çelësin në nyje.
- Gjindet intervali i search key.
- Përcillet link-u (rekursivisht).



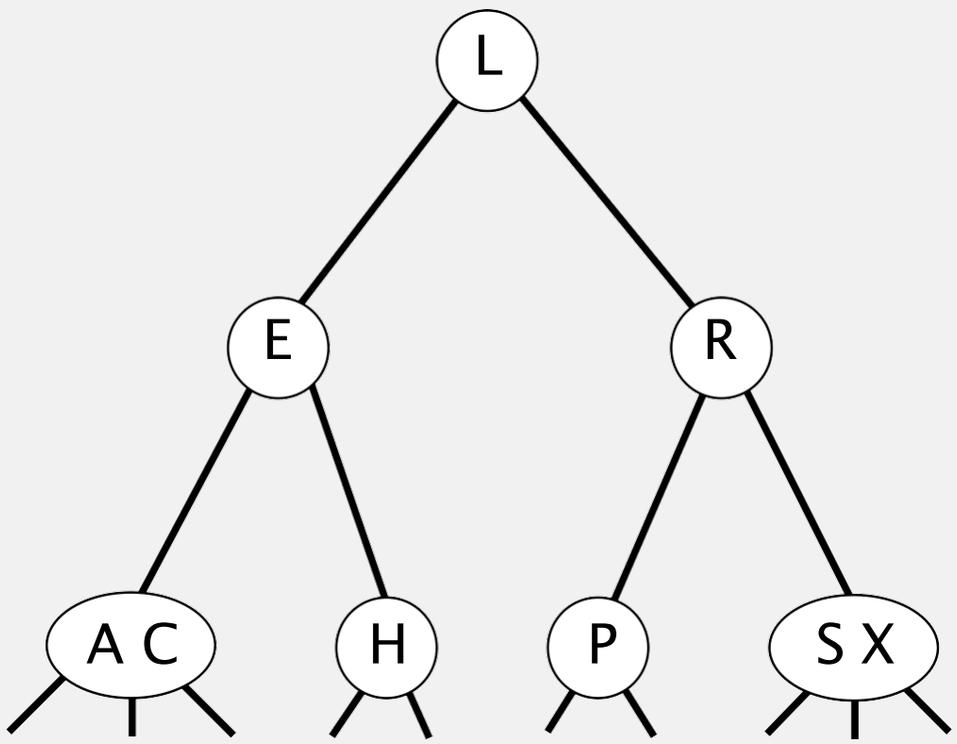
search për H

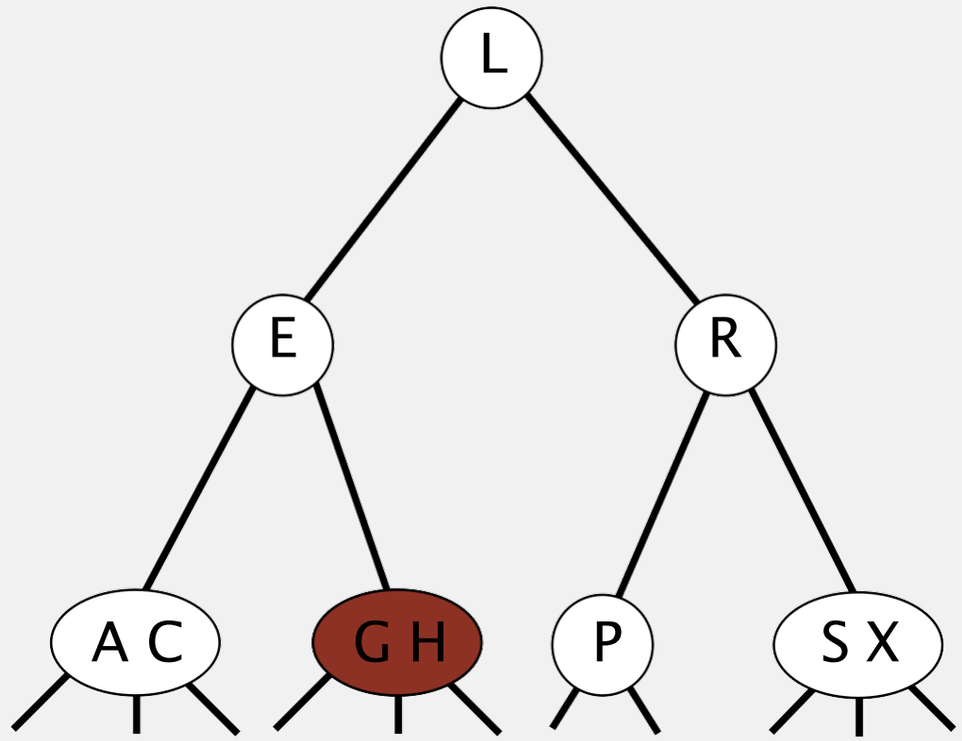


Insertion në një 2-3 tree

Insertion në një 2-node në fund.

insert G

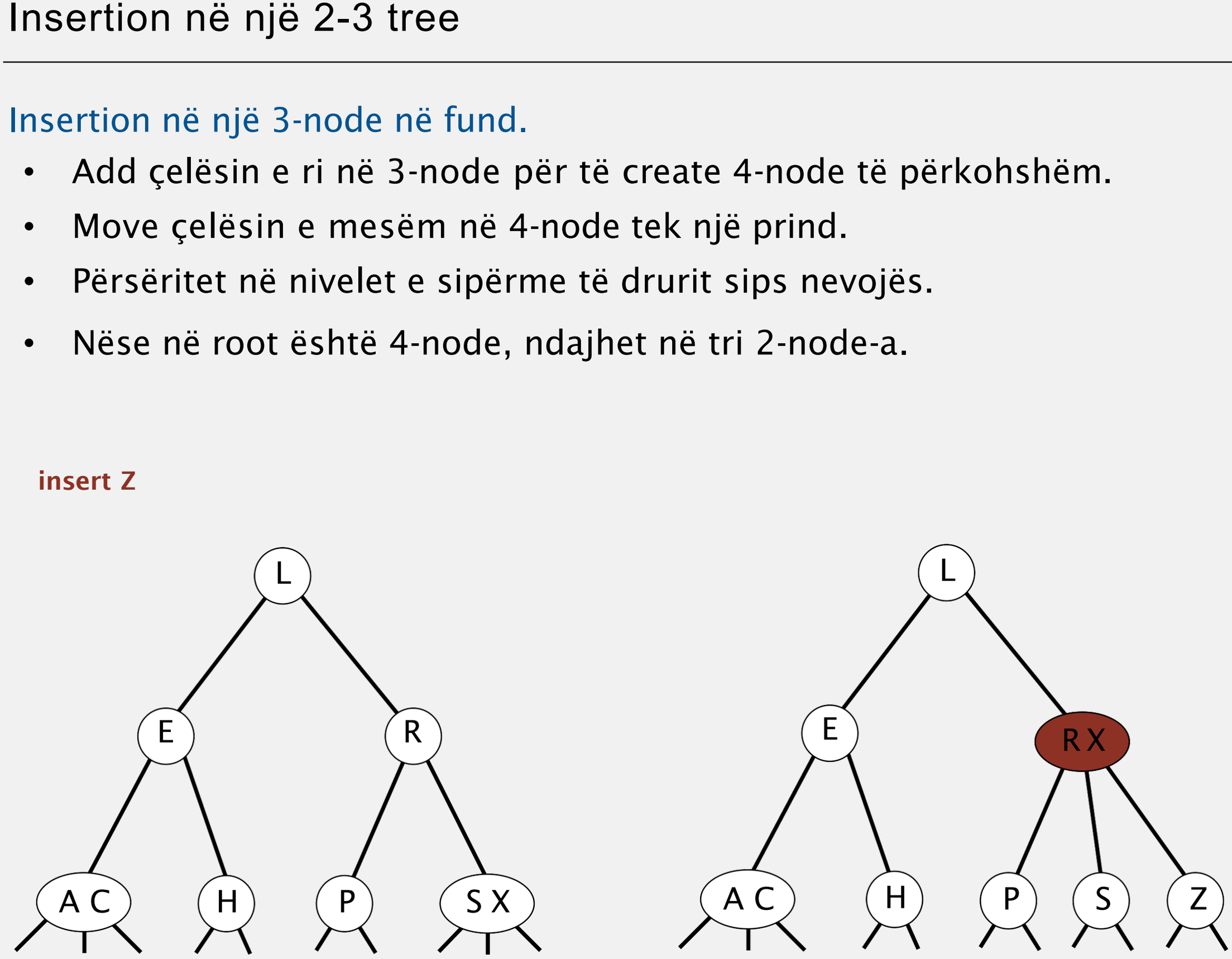




Insertion në një 2-3 tree

Insertion në një 3-node në fund.

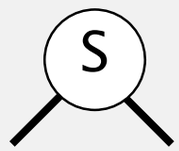
- Add çelësin e ri në 3-node për të create 4-node të përkohshëm.
- Move çelësin e mesëm në 4-node tek një prind.
- Përsëritet në nivelet e sipërme të drurit sips nevojës.
- Nëse në root është 4-node, ndajhet në tri 2-node-a.



2-3 tree demo - konstruktimi

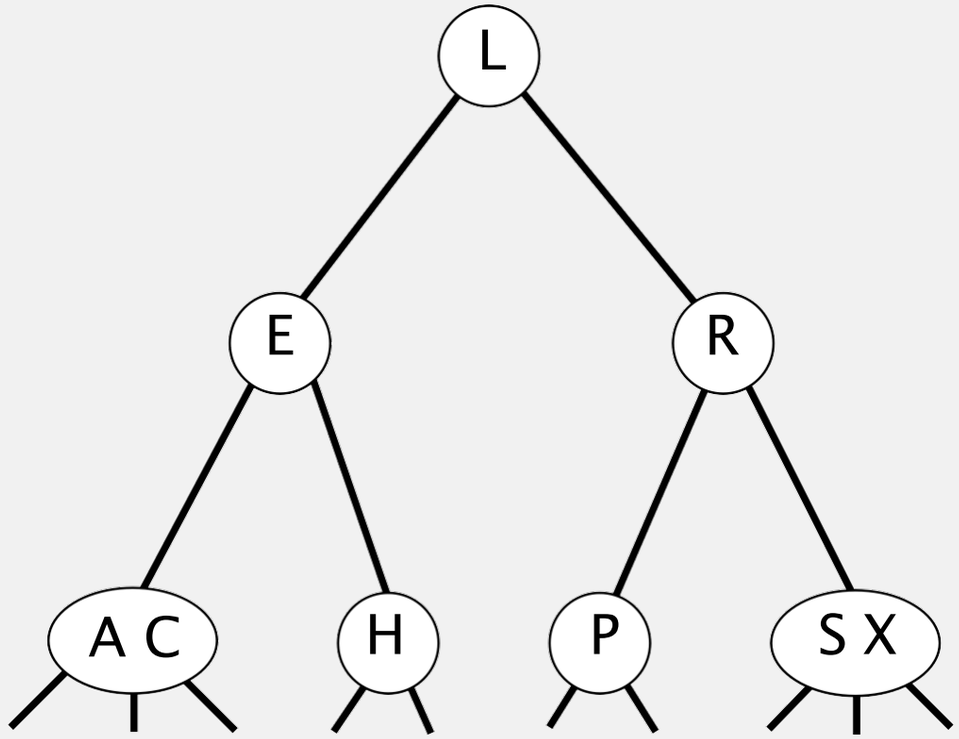
insert S





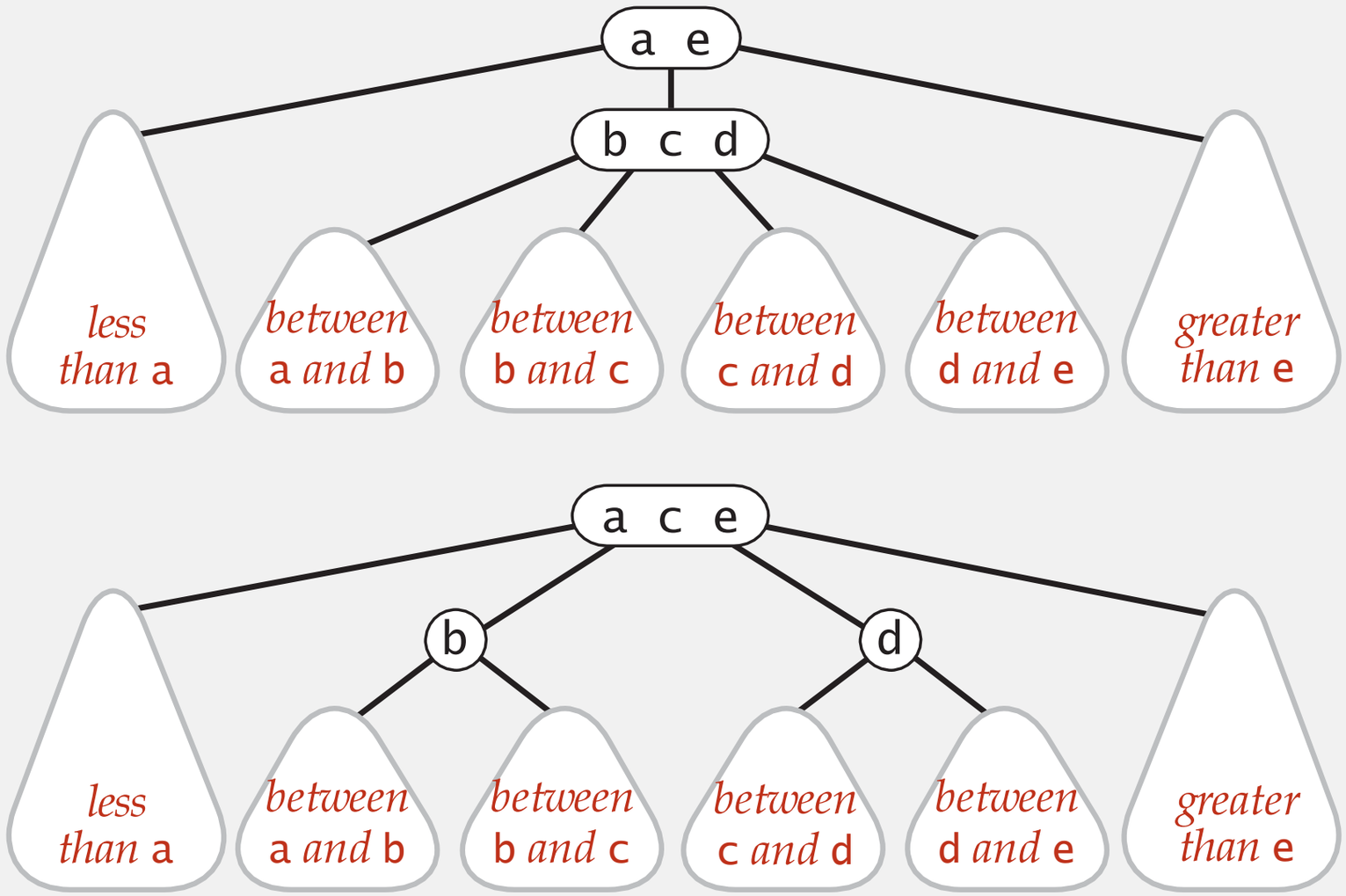
2-3 tree demo - konstruktimi

2-3 tree



Transformimet lokale në një 2-3 tree

Splitting (ndarja) e një 4-node është local transformation:
numër konstant i operacioneve.



Veçori të përgjithshme në një 2-3 tree

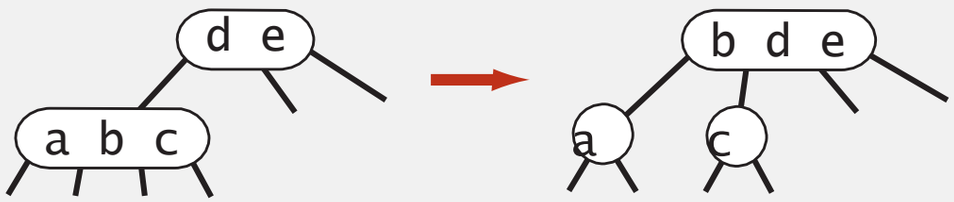
Vetitë. Druri ka rradhitje simetrike dhe balancim perfekt.

root



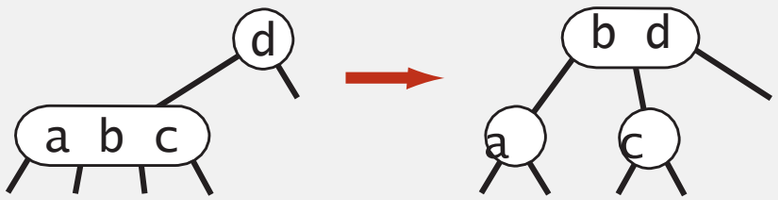
parent is a 3-node

left

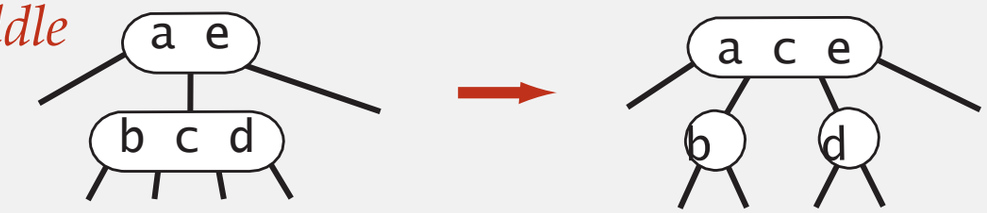


parent is a 2-node

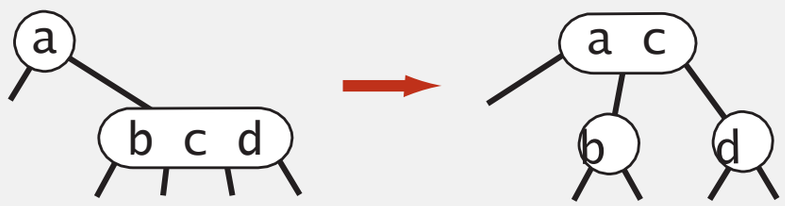
left



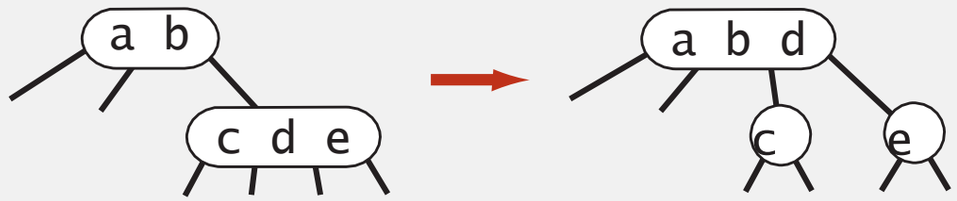
middle



right

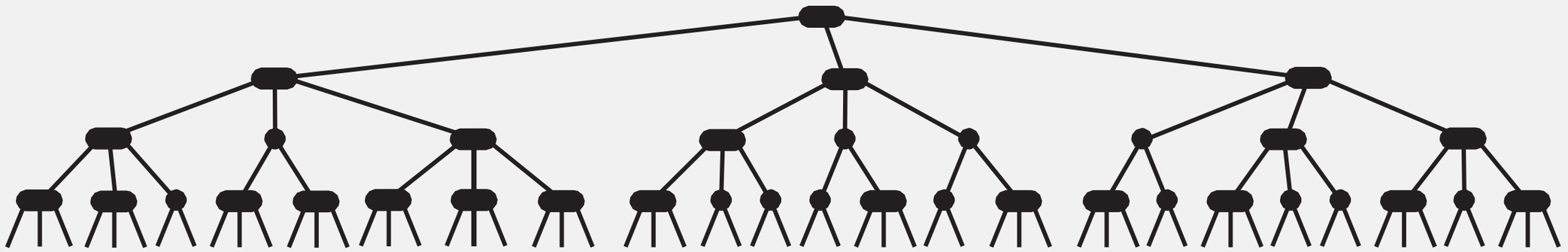


right



2-3 tree: performanca

Balancimi perfekt. Çdo rrugë prej root deri tek null link ka length të njejtë.

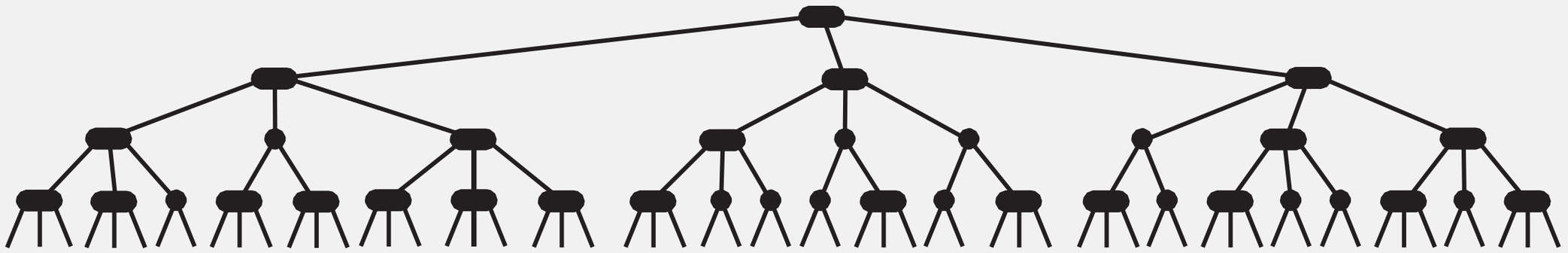


Lartësia e drurit.

- Worst case: e njejtë
- Best case: e njejtë

2-3 tree: performanca

Balancimi perfekt. Çdo rrugë prej root deri tek null link ka length të njejtë.



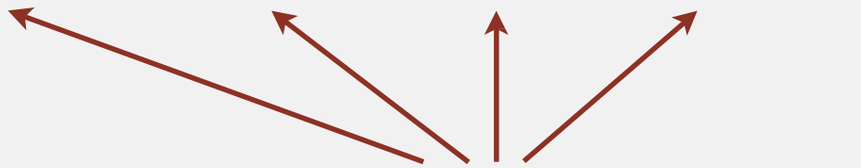
Lartësia e drurit.

- Worst case: $\lg N$. [all 2-nodes]
- Best case: $\log_3 N \approx .631 \lg N$. [all 3-nodes]

Përfundim. Garantohet performancë logaritmike për search dhe insert.

ST implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	✓	`compareTo()`





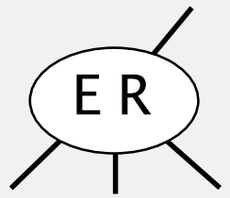
http://algs4.cs.princeton.edu

3.3 Balanced Search Trees

- 2-3 search trees
- red-black BSTs
- B-trees

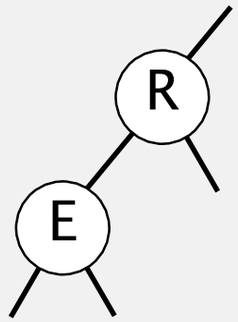
Implementimi i 2-3 trees me anë të binary trees?

Sfida. Si të paraqitet një 3 node?



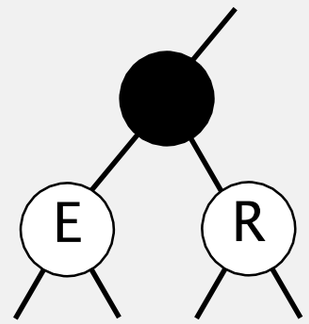
Qasja 1:BST i rregulltë

- Nuk mund të dihet a është një 3-node apo 2-node.
- Nuk mund të konvertohet prej BST në 2-3 tree.



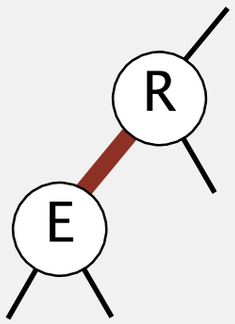
Qasja 2: BST i rregulltë me "glue" nodes.

- Shpenzon hapsirë, shpenzon një link.
- Kodi është i çrregulltë



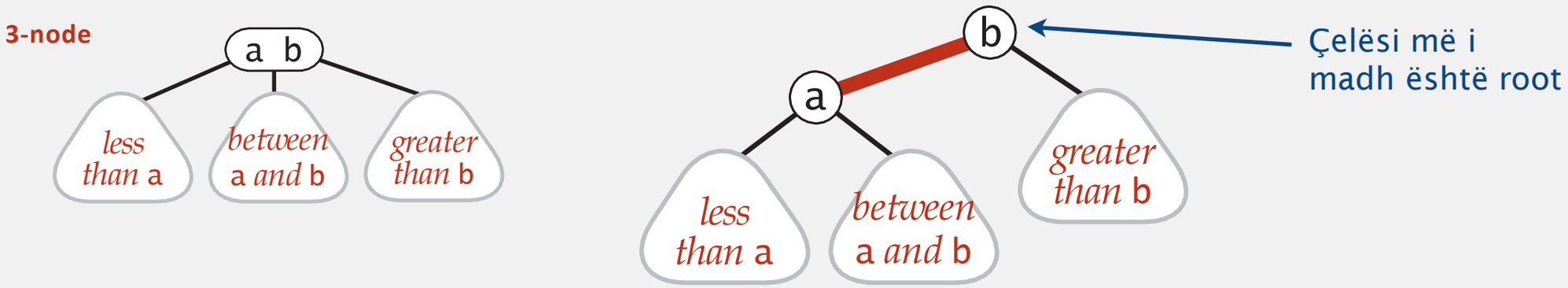
Qasja 3: BST i rregulltë me "glue" link-a të kuq.

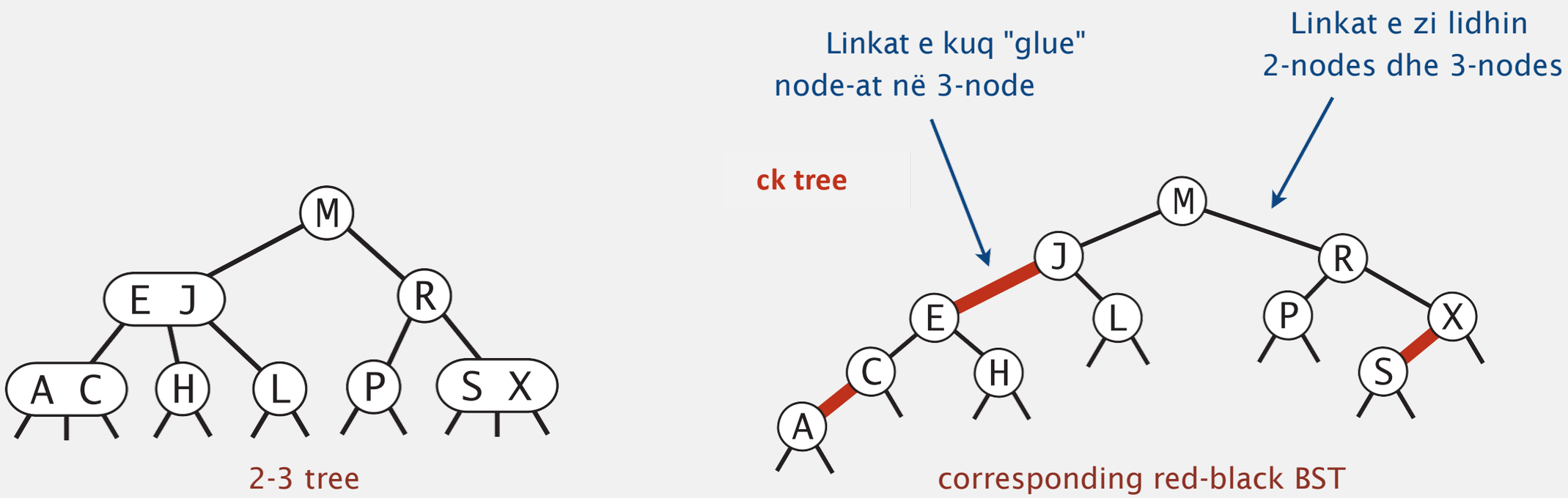
- Me përdorim të gjerë në praktikë.
- Rregullë arbitrare: link-at e kuq anojnë majtas.



Red-black BST (Guibas-Sedgewick 1979 dhe Sedgewick 2007)

1. Paraqitet 2-3 tree si një BST.
2. Përdoret link-u i anuar majtas si "glue" për 3-nodes.



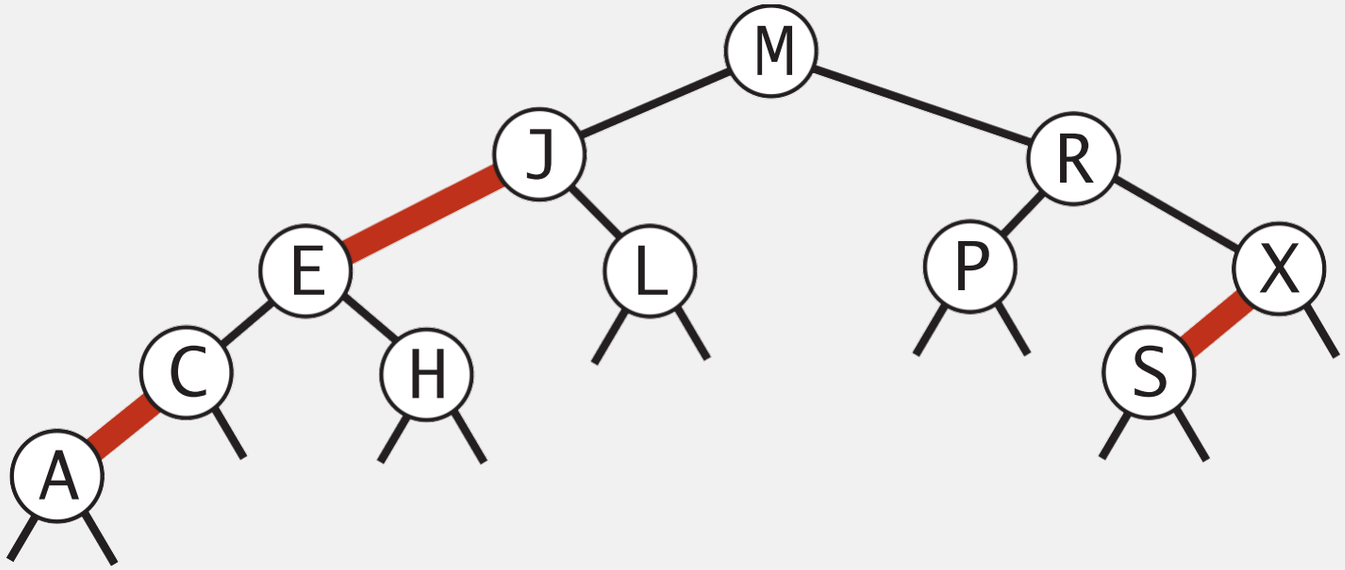


Rregullat për red-black trees

Një BST ashtu që:

- Asnjë nyje (node) nuk ka dy red link-a të lidhur për te.
- Çdo rrugë prej root tek null link-u ka një numër të njejtë të linkave të zi.
- Linkat e kuq anojnë majtas.

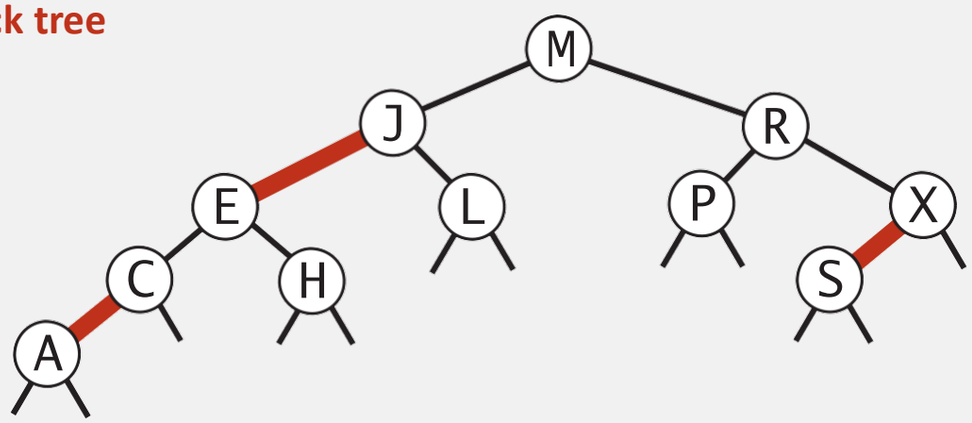
"perfect black balance"



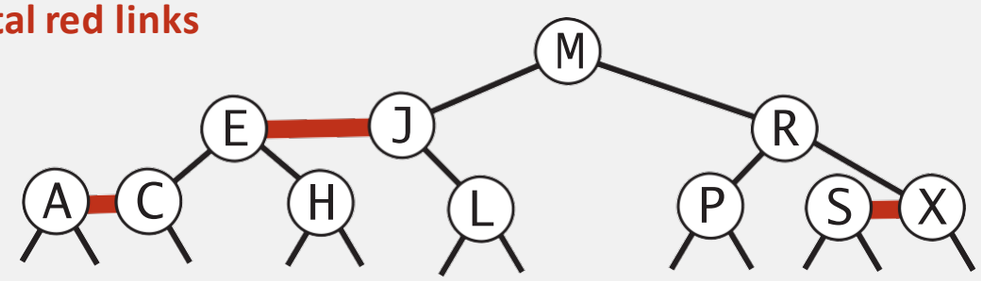
Red-black BST: 1-1 pasqyrimi me 2-3 trees

Vetitë. 1-1 korespondon në mes të 2-3 dhe Red Black tree.

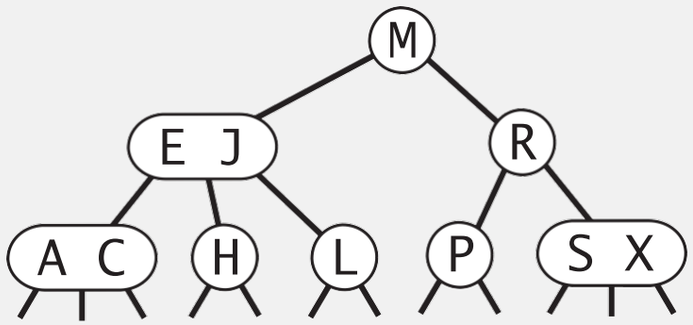
red-black tree



horizontal red links



2-3 tree

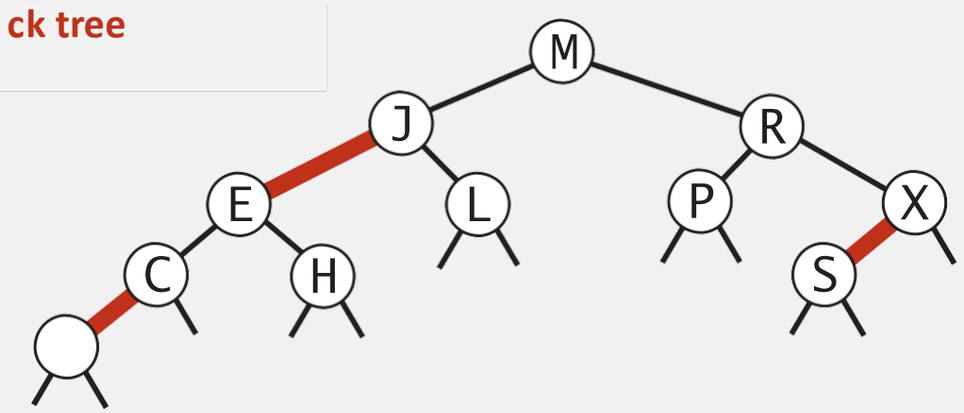


Search implementimi për red-black BST

Observim. Search është njësoj sikur për BST të thjeshtë (injorohet ngjyra).

Ekzekutohet më shpejtë
Sepse është më i balancuar

```
public Val get(Key key)
{
   Node x = root;
   while (x != null)
   {
      int cmp = key.compareTo(x.key);
      if      (cmp  < 0) x = x.left;
      else if (cmp  > 0) x = x.right;
      else if (cmp == 0) return x.val;
   }
   return null;
}
```



Vërejtje. Shumica e operacioneve të tjera (p.sh., floor, iteration, selection) gjithashtu janë identike.

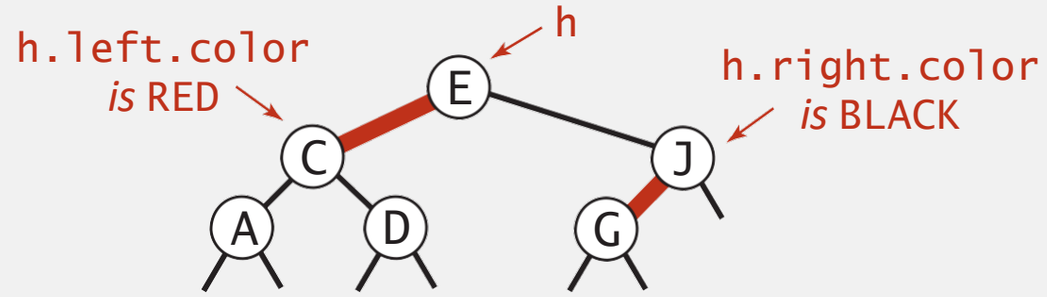
Red-black BST - paraqitja

```
private static final boolean RED   = true;
private static final boolean BLACK = false;

private class Node
{
   Key key;
   Value val;
   Node left, right;
   boolean color;   // color of parent link
}

private boolean isRed(Node x)
{
   if (x == null) return false;
   return x.color == RED;
}
```

null link-at janë të zi

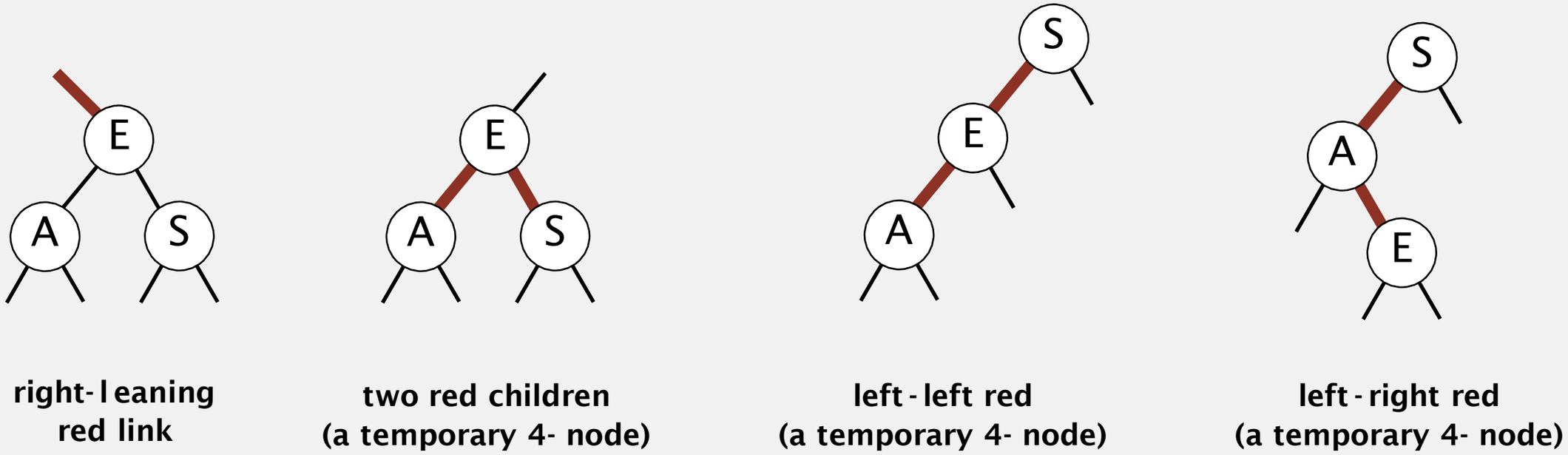


Insertion në një red black tree: overview

Strategjia bazë. Mbahet 1-1 korespondenca me 2-3 trees.

Përgjatë operacioneve, të mbahet:

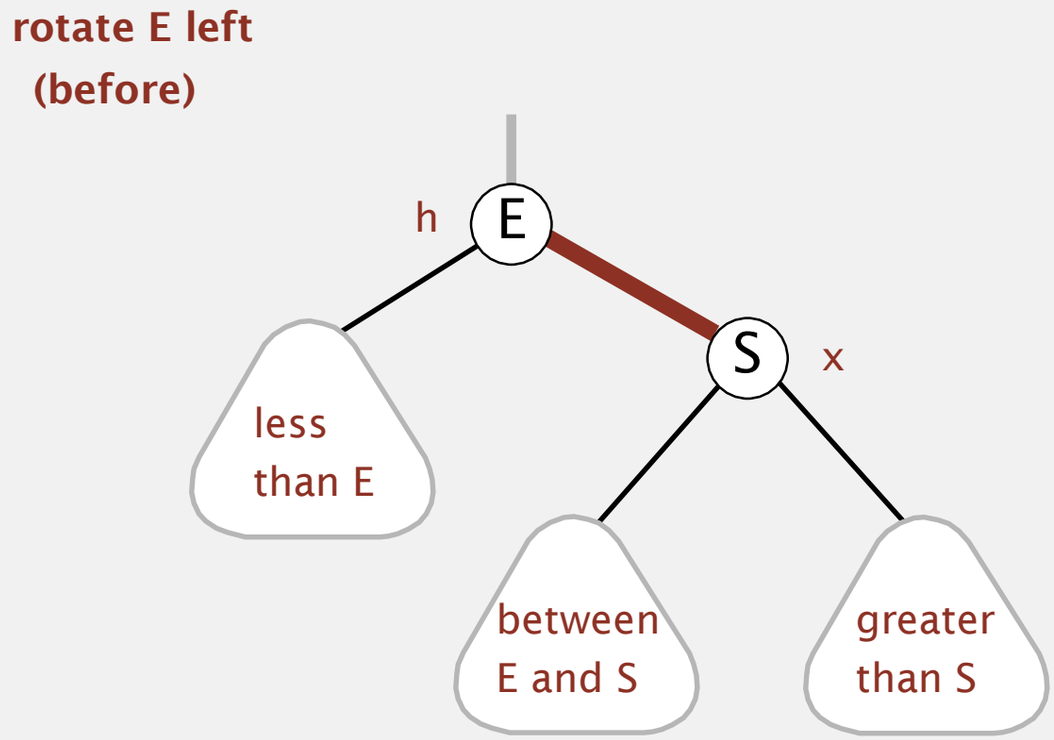
- Rradhitja simetrike.
- Balanci perfekt – druri është i zi.



Vërejtje? Aplikohen operacionet elementare për BST kuq e zi:
Rrotullimet (rotation) dhe ndryshimi i ngjyrave (flip colors).

Operacionet elementare për BST kuq e zi

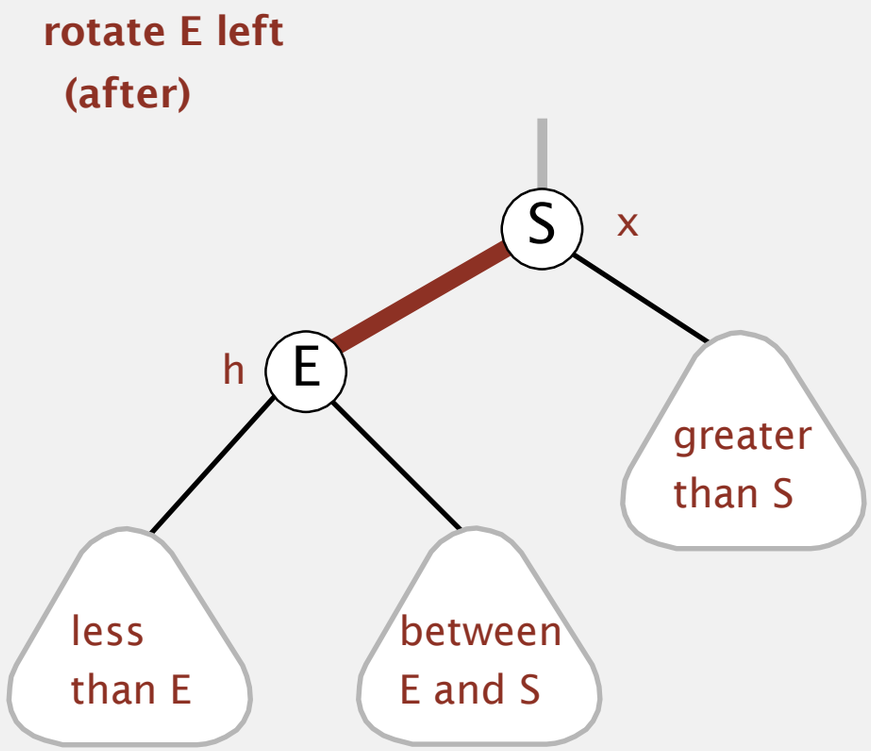
Rrotullimi i majtë. Orientohet linku i kuq (i përkohshëm) i anuar djathtë që të anojë në të majtë.




```
private Node rotateLeft(Node h)
{
   assert isRed(h.right);
   Node x = h.right;
   h.right = x.left;
   x.left = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Operacionet elementare për BST kuq e zi

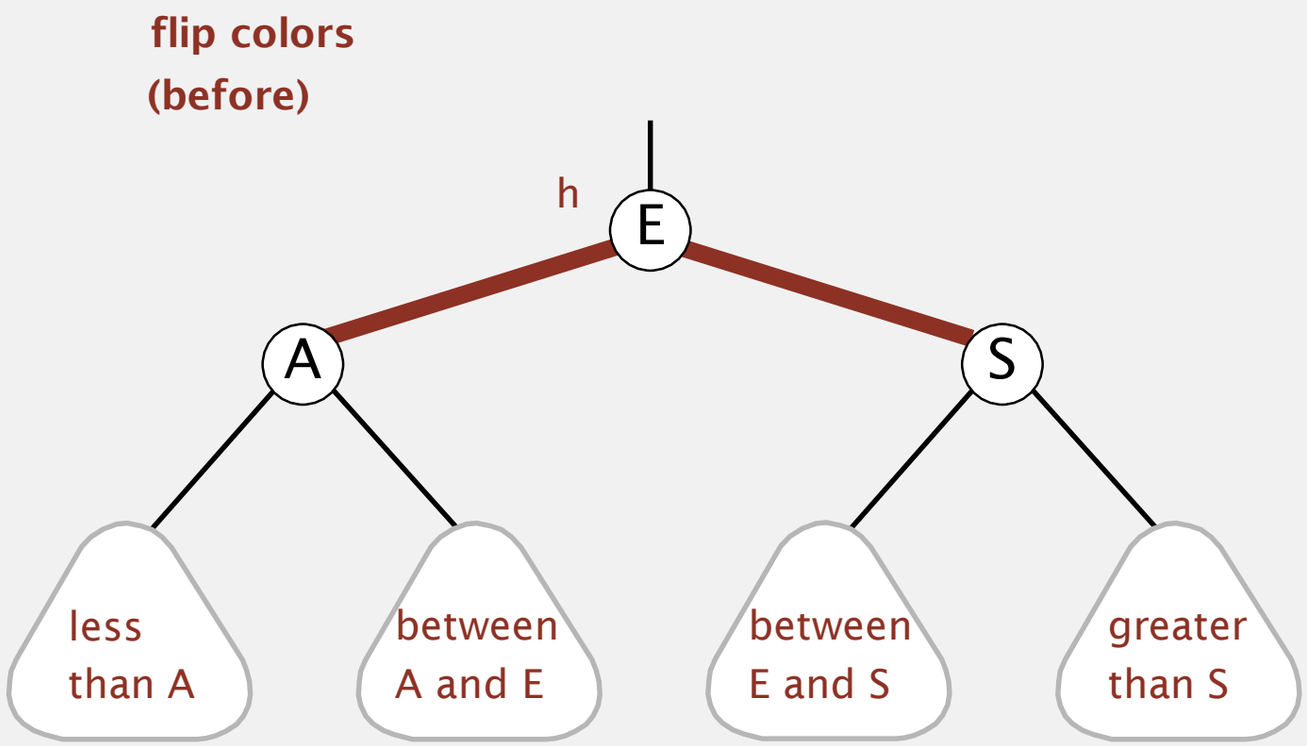
Rrotullimi i majtë. Orientohet linku i kuq (i përkohshëm) i anuar djathtë që të anojë në të majtë.




```
private Node rotateLeft(Node h)
{
   assert isRed(h.right);
   Node x = h.right;
   h.right = x.left;
   x.left = h;
   x.color = h.color;
   h.color = RED;
   return x;
}
```

Operacionet elementare për BST kuq e zi

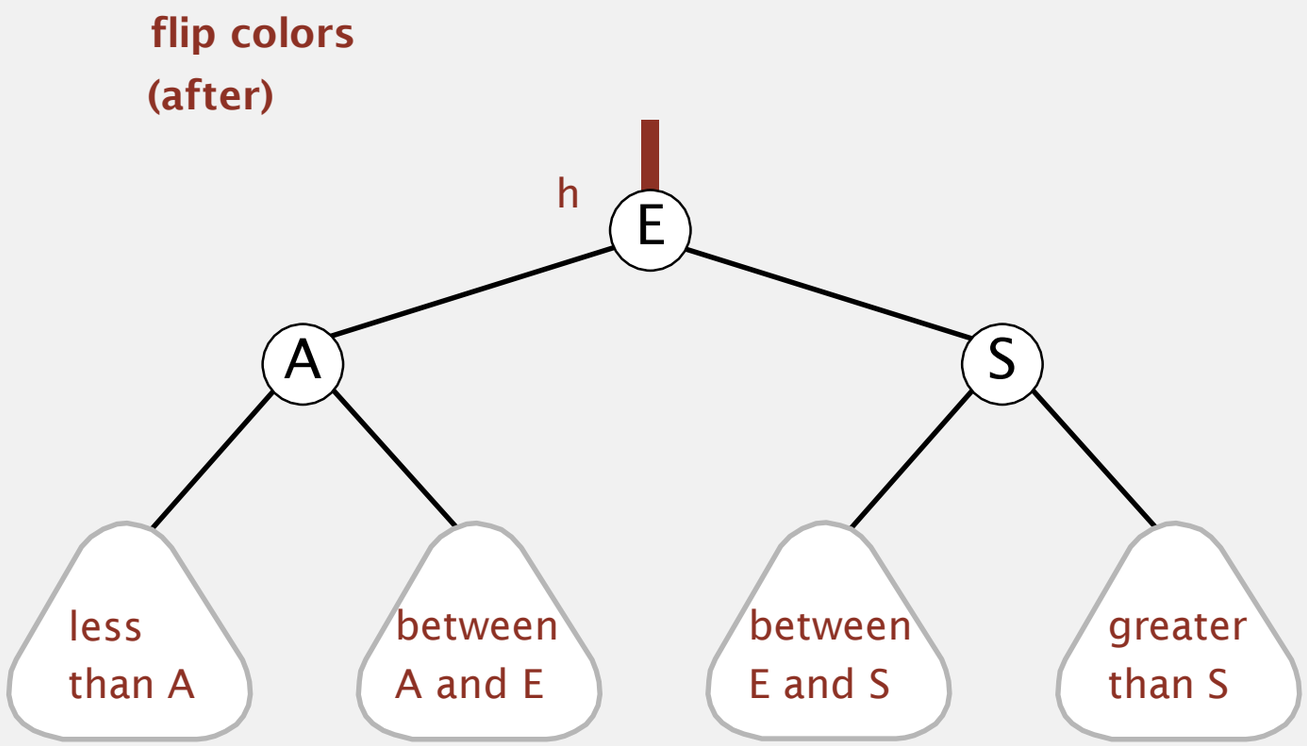
Color flip. Ringjyoset druri kur ka nyje me dy fëmijë të kuq.




```
private void flipColors(Node h)
{
   assert !isRed(h);
   assert isRed(h.left);
   assert isRed(h.right);
   h.color = RED;
   h.left.color = BLACK;
   h.right.color = BLACK;
}
```

Operacionet elementare për BST kuq e zi

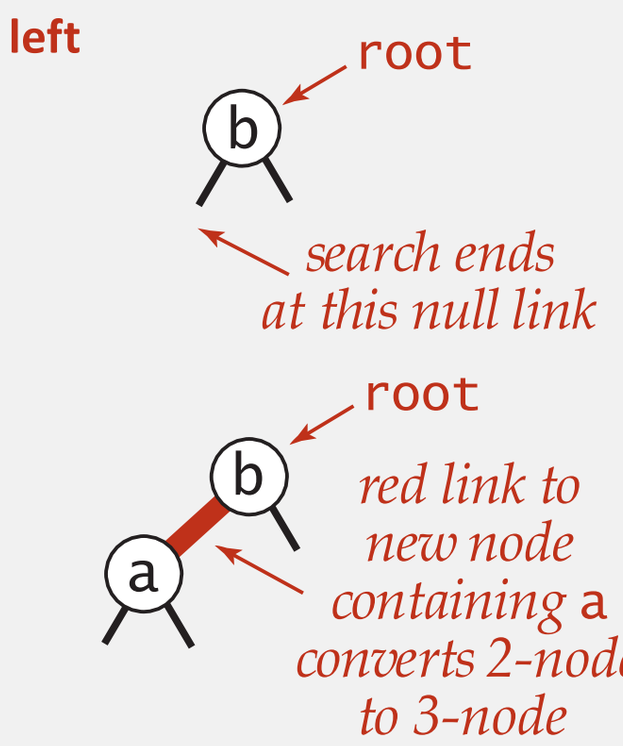
Color flip. Ringjyoset druri kur ka nyje me dy fëmijë të kuq.

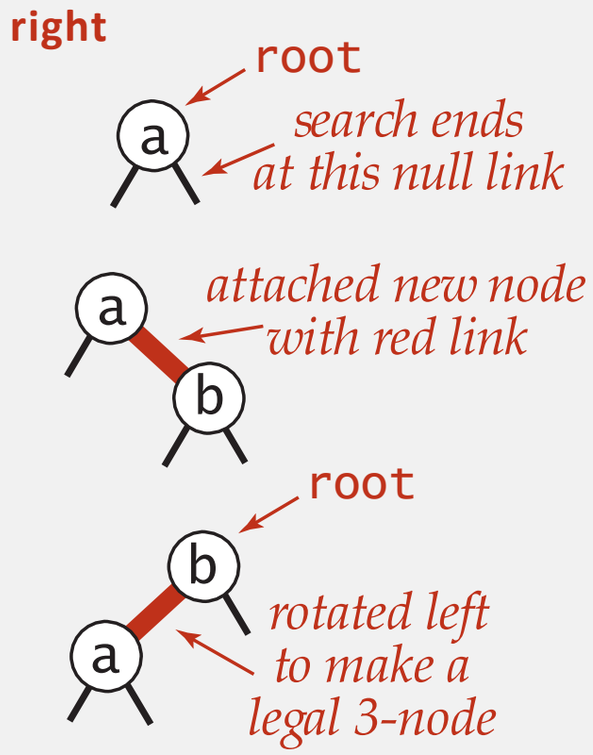



```
private void flipColors(Node h)
{
   assert !isRed(h);
   assert isRed(h.left);
   assert isRed(h.right);
   h.color = RED;
   h.left.color = BLACK;
   h.right.color = BLACK;
}
```

Insertimi në një dry kuq e zi (Red Black tree)

Warmup 1. Insert into a tree with exactly 1 node.

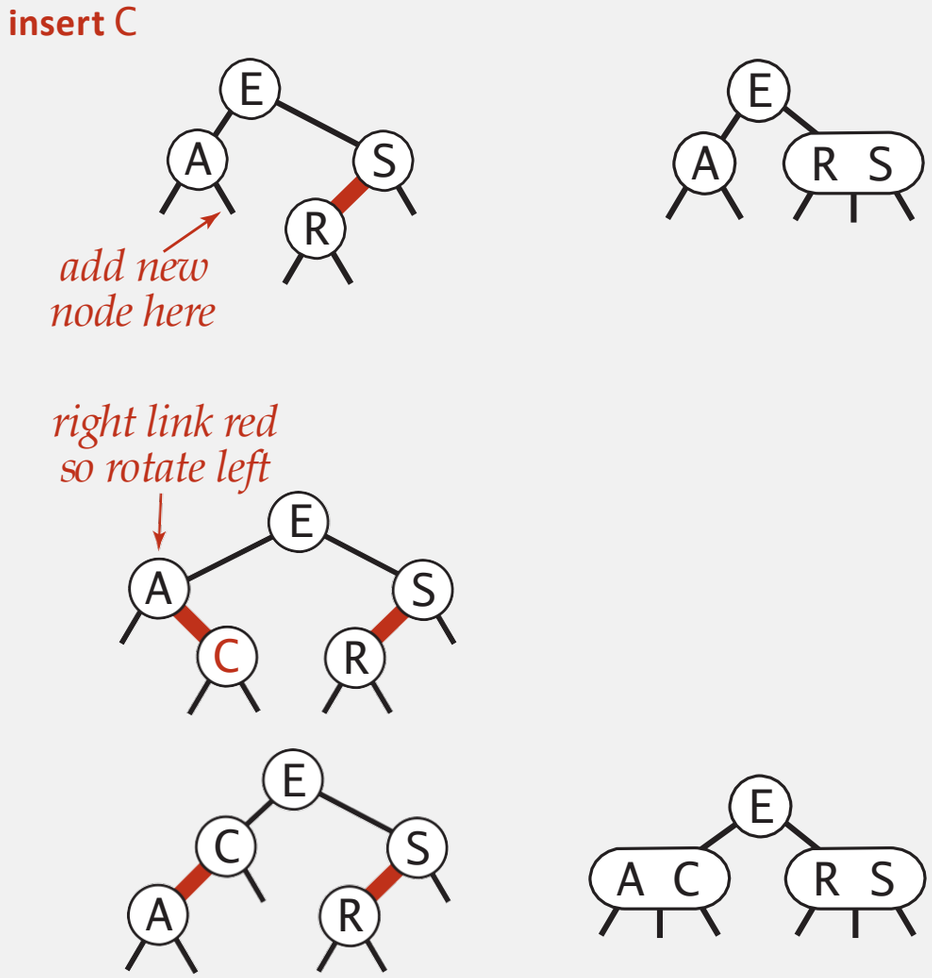




Insertimi në një dry kuq e zi (Red Black tree)

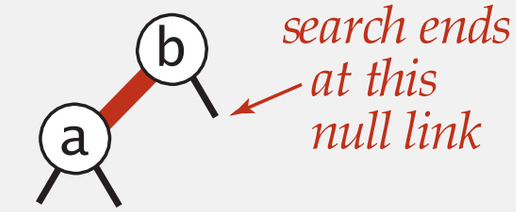
Rasti 1. Insertimi në një 2-node në fund të drurit

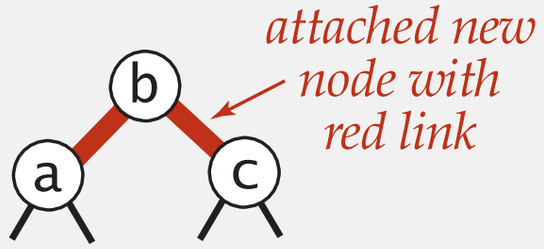
- Insertimi sikur në BST; link-u i ri ngjyroset kuq. ← to maintain symmetric order and perfect black balance
- Nëse link-u i ri është i djathtë, rotate left. ← to fix color invariants

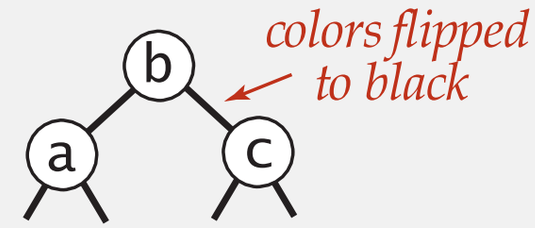


Insertimi në një dry kuq e zi (Red Black tree)

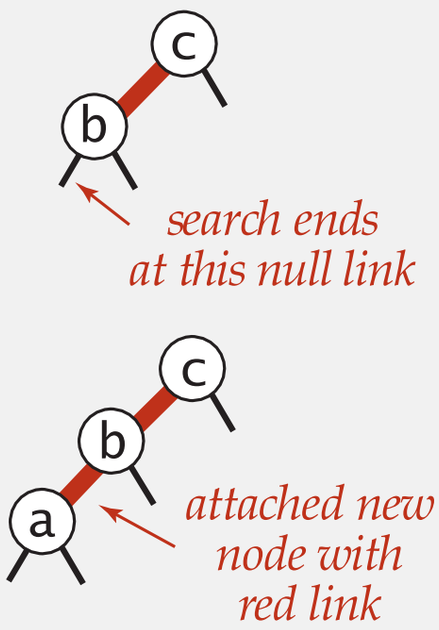
larger

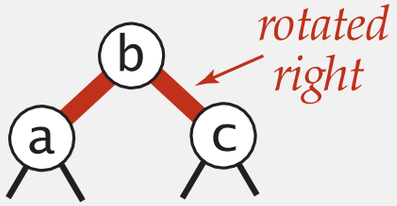


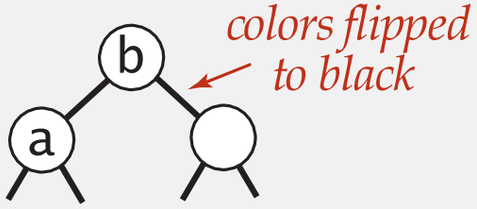




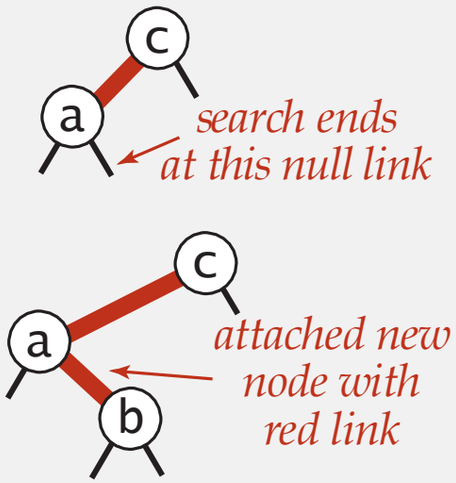
smaller

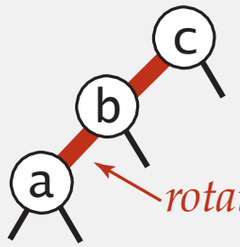


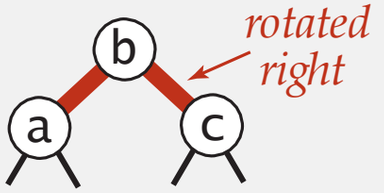


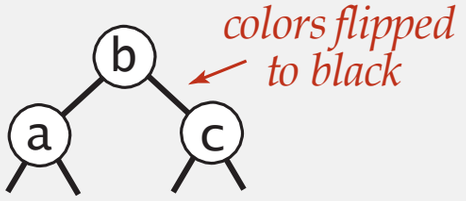


between









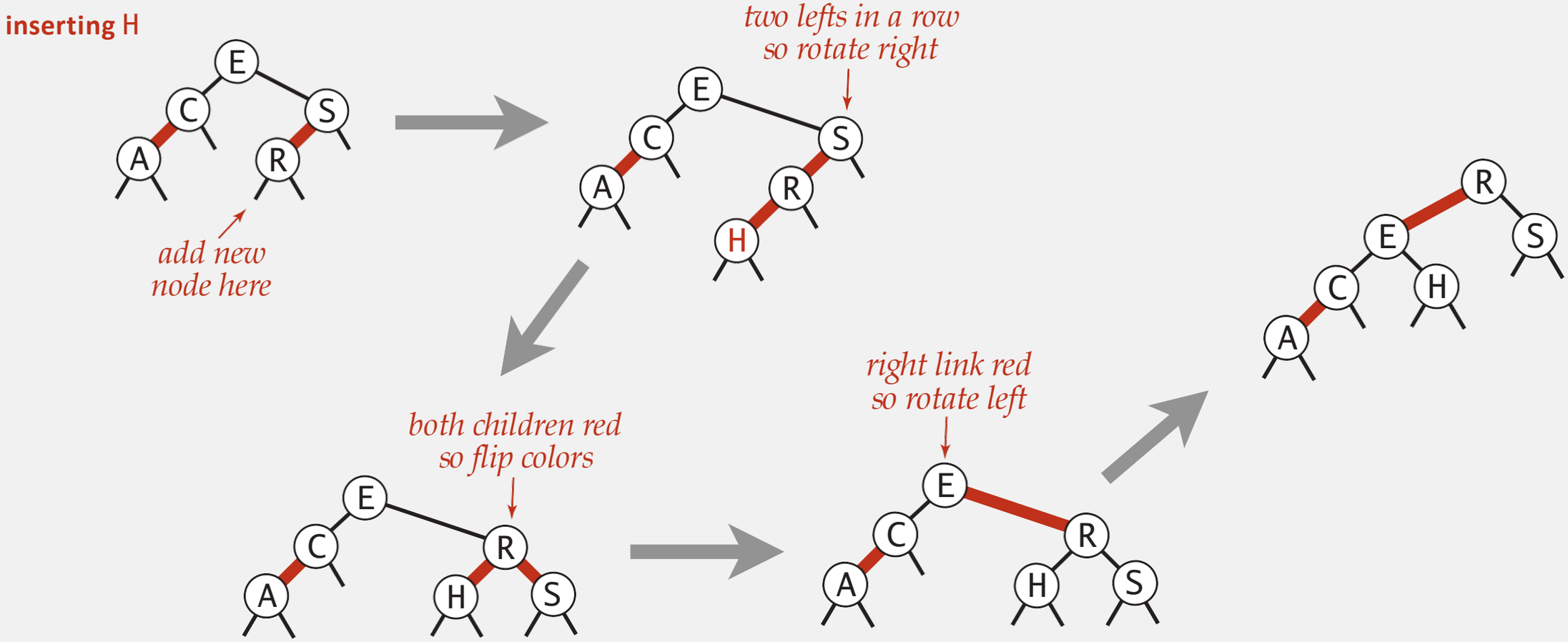
Insertimi në një dry kuq e zi (Red Black tree)

Rasti 2. Insertimi në një 3-node në fund të drurit

- Insertimi sikur në BST; link-u i ri ngjyroset kuq.
- Rotate për balancim 4-node-in (nëse duhet).
- Flip colors që të bartet link-u një nivel më lartë.
- Rotate që të anojë majtas (nëse duhet).

← to maintain symmetric order and perfect black balance

← to fix color invariants



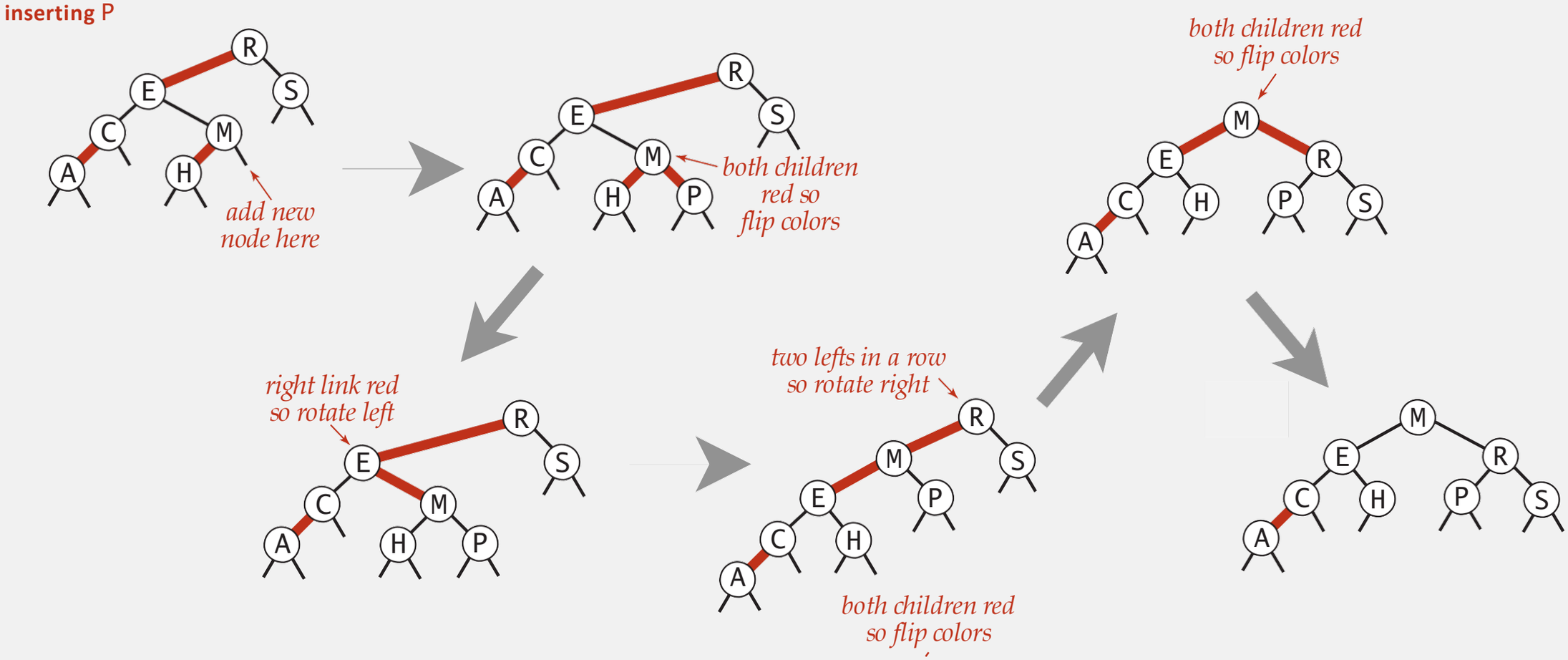
Insertimi në një dry kuq e zi (Red Black tree)

Rasti 2. Insertimi në një 3-node në fund të drurit

- Insertimi sikur në BST; link-u i ri ngjyroset kuq..
- Rotate për balancim 4-node-in (nëse duhet).
- Flip colors që të bartet link-u një nivel më lartë.
- Rotate që të anojë majtas (nëse duhet).
- Përsëritet rasti 1 ose rasti 2 në dru (nëse duhet).

to maintain symmetric order and perfect black balance

to fix color invariants



Red-black BST konstruktimi - demo

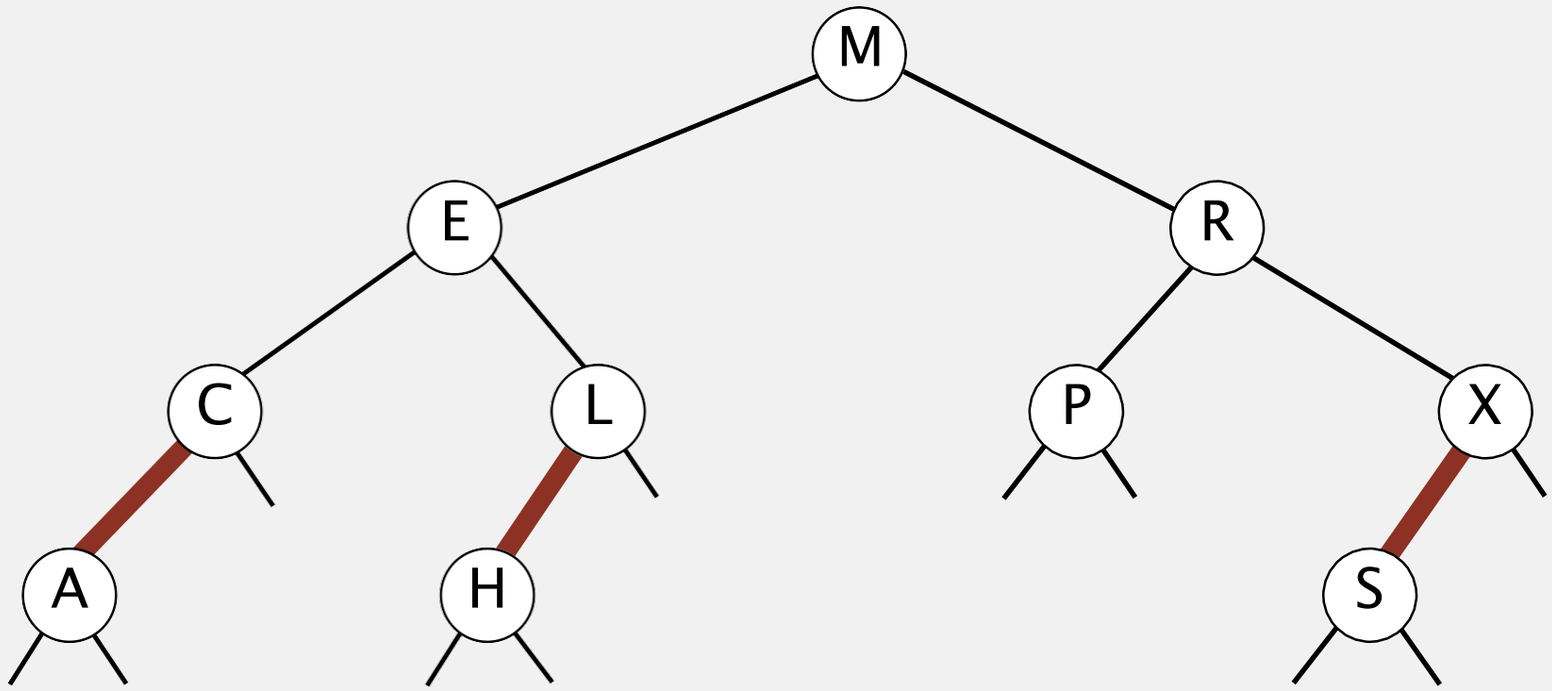
insert S





Red-black BST konstruktimi - demo

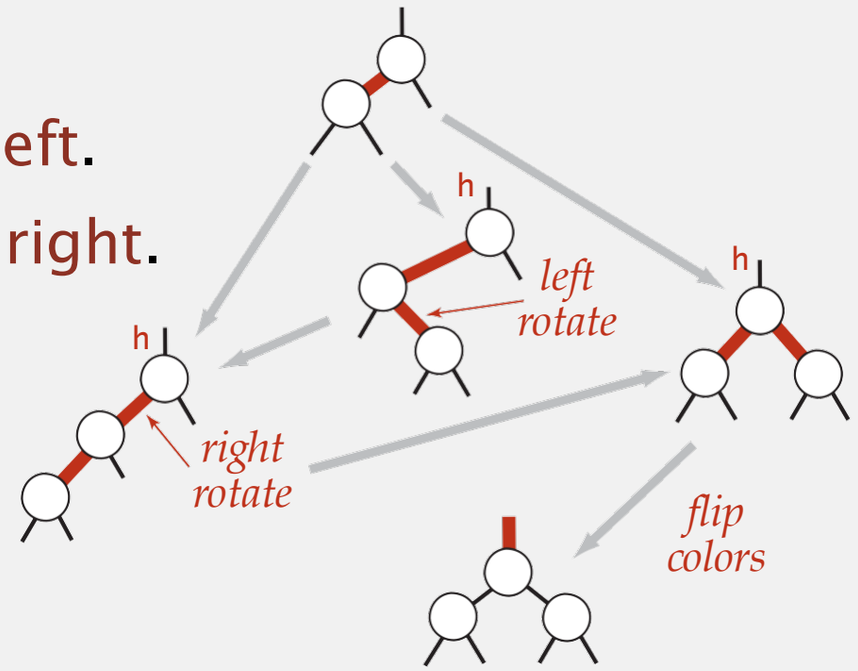
red-black BST



Insertimi në një dry kuq e zi: Java implementimi

Kodi i njejtë për të gjitha rastet.

- Fëmiu i kuq djathtas, fëmiu i zi majtas: rotate left.
- Dy fëmijë të kuq majtas njëri pas tjetrit: rotate right.
- Dy femijët e kuq: flip colors.




```
private Node put(Node h, Key key, Value val)
{
   if (h == null) return new Node(key, val, RED);
   int cmp = key.compareTo(h.key);
   if      (cmp  < 0) h.left  = put(h.left,  key, val);
   else if (cmp  > 0) h.right = put(h.right, key, val);
   else if (cmp == 0) h.val = val;

   if (isRed(h.right) && !isRed(h.left))
   if (isRed(h.left)  && isRed(h.left.left))
   if (isRed(h.left)  && isRed(h.right))

   return h;
}
```

insert at bottom (and color it red)

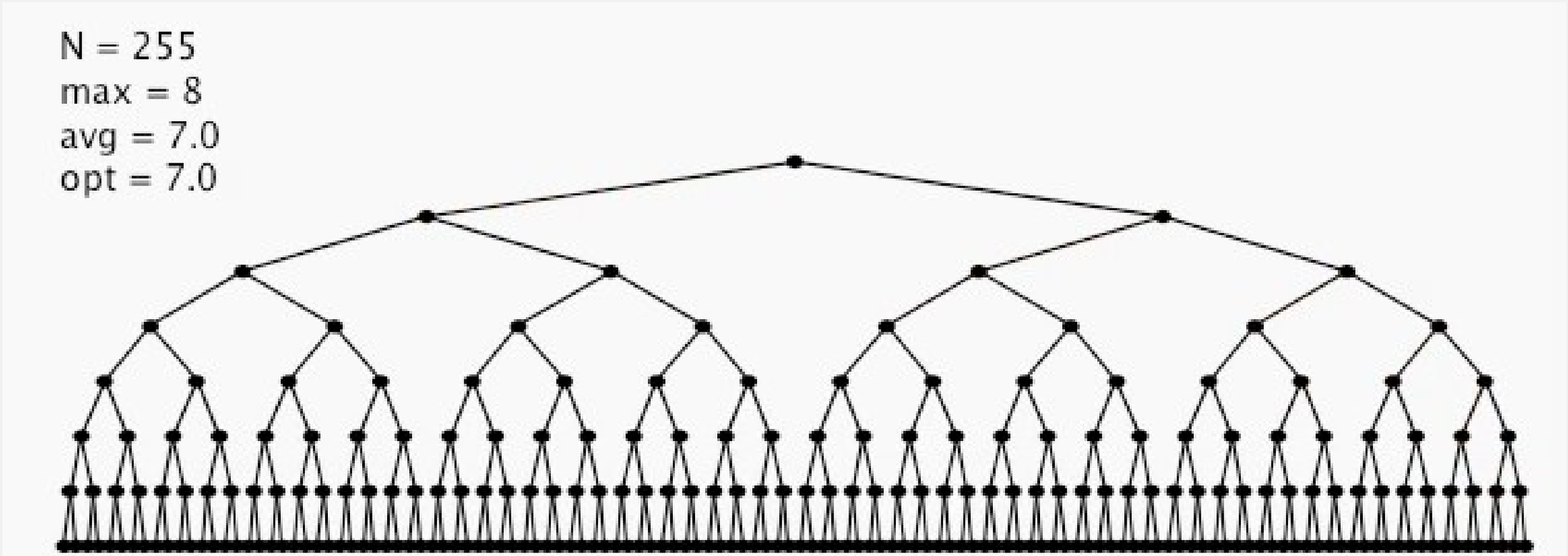
lean left

balance 4-node

split 4-node

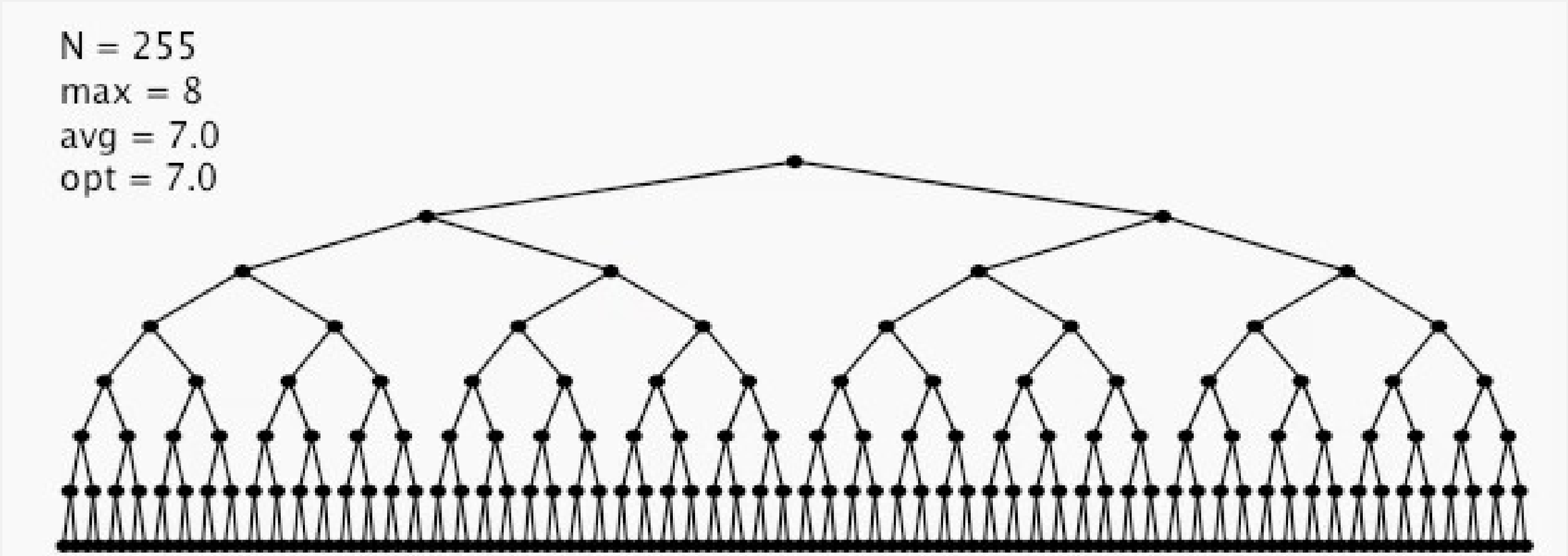
only a few extra lines of code provides near-perfect balance

Insertimi në një dry kuq e zi: vizualizim



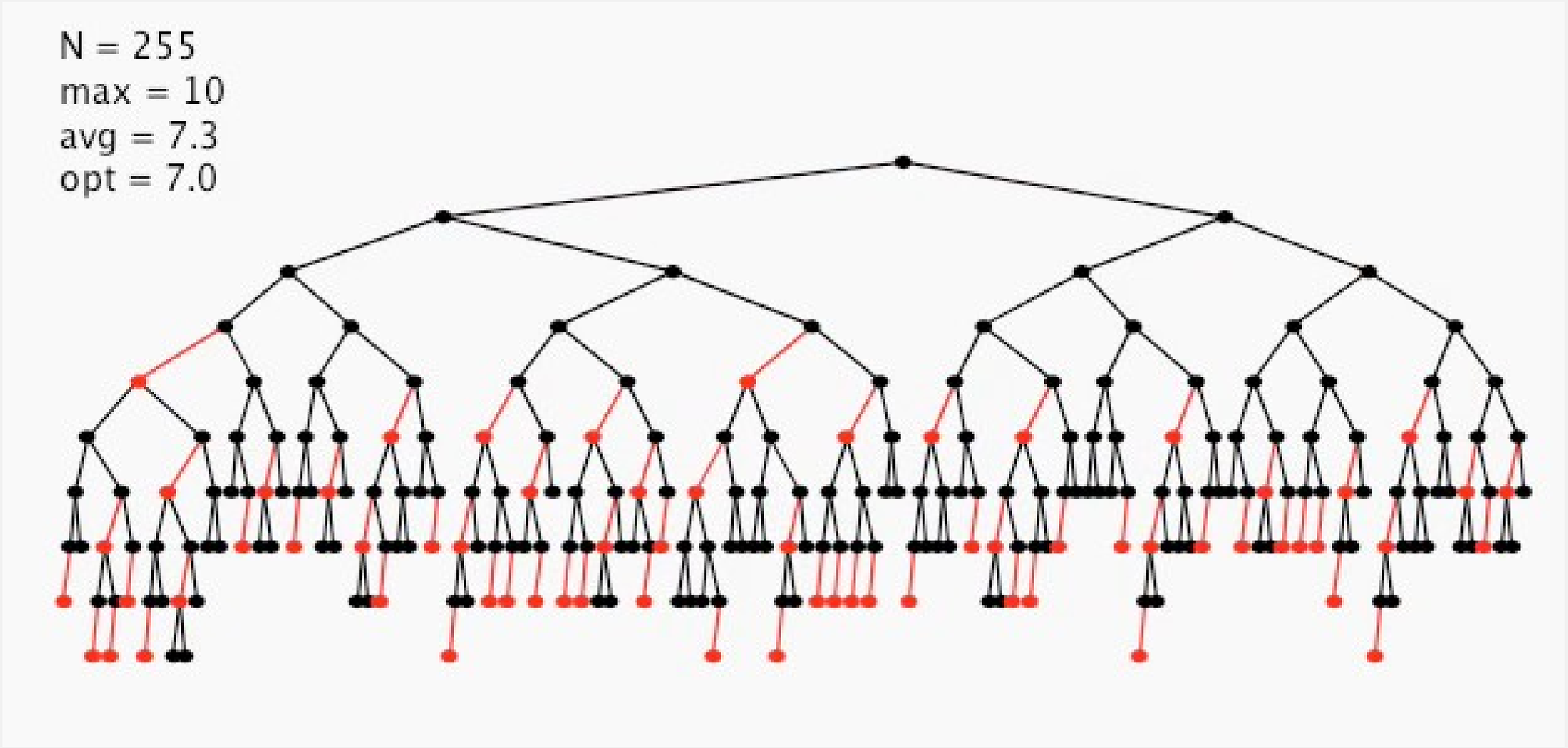
255 insertions in ascending order

Insertimi në një dry kuq e zi: vizualizim



255 insertions in descending order

Insertimi në një dry kuq e zi: vizualizim

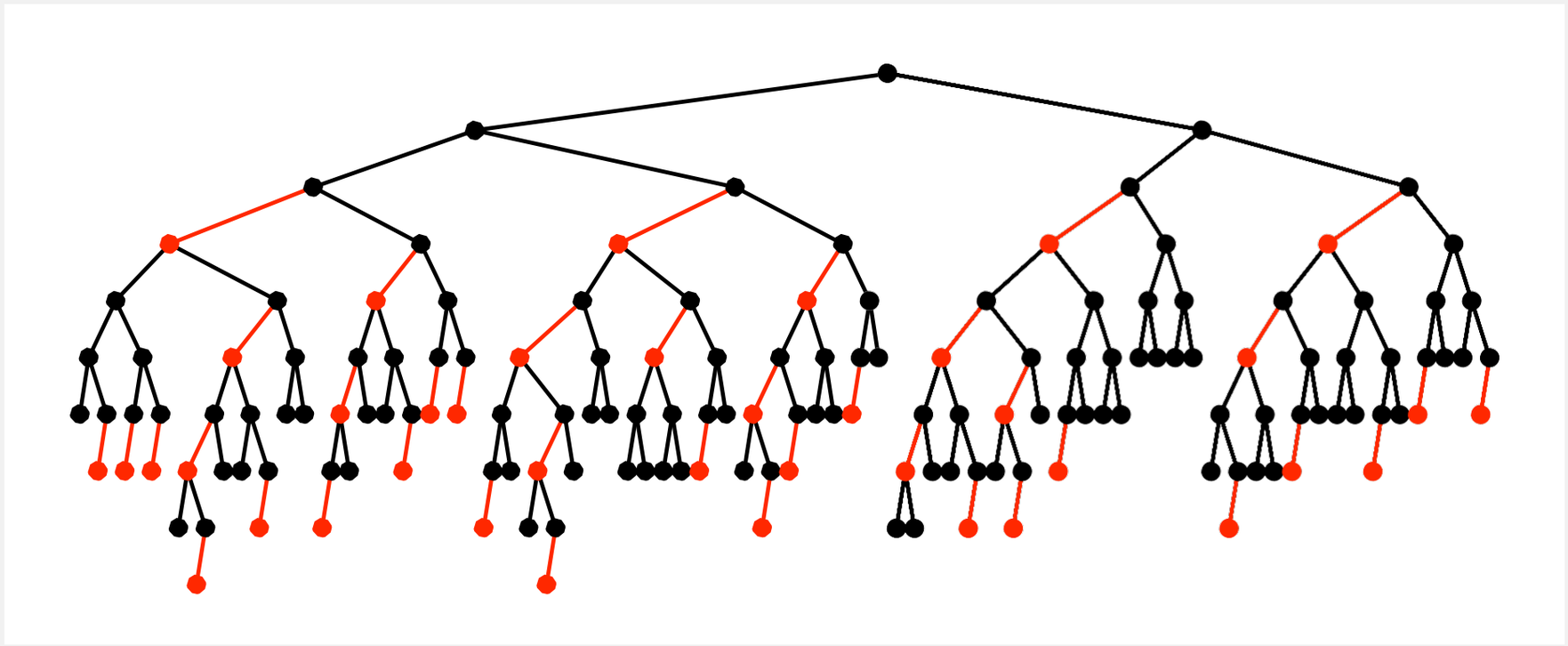


255 random insertions

Balansimi në një dry kuq e zi

Pohim. Lartësia e drurit është $\leq 2 \lg N$ në rastin më të keq.

- Çdo rrugë orej root tek null link-u ka numër të njejtë të linkave të zi.
- Kurrë nuk ka dy link-a të kuq në seri (të njëpasnjëshëm).



Vëçori. Lartësia e drurit është ~ $1.0 \lg N$ në rastin tipik

ST implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	✓	`compareTo()`
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$ *	$1.0 \lg N$ *	$1.0 \lg N$ *	✓	`compareTo()`

* exact value of coefficient unknown but extremely close to 1